



FACULTAD DE INGENIERÍA

Machine Learning en Producción

Obligatorio: Predicción de ratings de películas IMDb

2026

Alumnos:

Pablo Durán - 270956

Marcos Mussio - 362839

Natalia Campiglia - 349251



[mlp-obligatorio-imdb](#)

Última actualización: 9/07/2026

Índice

1. Resumen	4
2. Objetivo del proyecto	4
3. Arquitectura general	5
4. Recolección de datos	6
4.1. Ingesta de datos	7
4.2. Almacenamiento y versionado de datos	8
4.3. ETL	9
4.4. Logs y diagnóstico del proceso	11
4.5. Limitantes	12
5. Entrenamiento	12
5.1. Feature engineering	13
5.1.1. Feature engineering del entrenamiento actual	14
5.2. Entrenamiento del modelo	14
5.3. Publicación en W&B	15
5.4. Prevención de data leakage y training-serving skew	16
5.4.1. Data leakage	16
5.4.2. Training-serving skew	17
5.5. Trazabilidad de ML	18
6. Servicio/API	18
6.1. API de inferencia	19
6.2. Predicciones en landing page	19
6.3. Predicciones batch	20
6.4. Containerización	21
7. Infraestructura y monitoreo	21
7.1. Infraestructura como código	21

7.2. Gestión de secretos	22
7.3. Observabilidad y monitoreo	22
8. Trabajo pendiente	23
9. Limitaciones del proyecto y arquitectura ideal	23
9.1. Limitantes	23
9.2. Arquitectura ideal	24
10. Conclusiones	26

1. Resumen

El proyecto implementa una solución de Machine Learning en Producción para predecir el rating de películas de IMDb. La solución cubre el ciclo completo desde la recolección de datos hasta el despliegue de una aplicación web en AWS.

El flujo principal es:

1. Recolectar datos de películas y reviews desde IMDb.
2. Persistir datos crudos en S3 en formato Parquet.
3. Ejecutar un proceso ETL para construir datasets de entrenamiento e inferencia.
4. Entrenar, evaluar y registrar un modelo de predicción.
5. Exponer la solución mediante una API FastAPI containerizada.
6. Desplegar la aplicación en AWS ECS Fargate usando Terraform.

La solución busca aplicar conceptos de MLOps vistos en clase: automatización de pipelines, versionado de datos, separación entre datos crudos y procesados, tracking de modelos, model registry, despliegue reproducible, gestión de secretos y monitoreo básico. Estas prácticas son relevantes porque los sistemas de Machine Learning suelen acumular deuda técnica no solo en el modelo, sino también en los datos, pipelines, configuración y serving [4, 3].

2. Objetivo del proyecto

El objetivo es construir una aplicación capaz de estimar el rating esperado de una película a partir de atributos disponibles en IMDb, como año, duración, metaspcore, géneros, director, elenco principal y señales derivadas de reviews de usuarios.

Además del modelo predictivo, el foco del obligatorio está en construir una arquitectura productiva alrededor del modelo. Por eso el proyecto no se limita al entrenamiento, sino que incluye

ingesta de datos, procesamiento, almacenamiento, despliegue cloud e infraestructura como código.

3. Arquitectura general

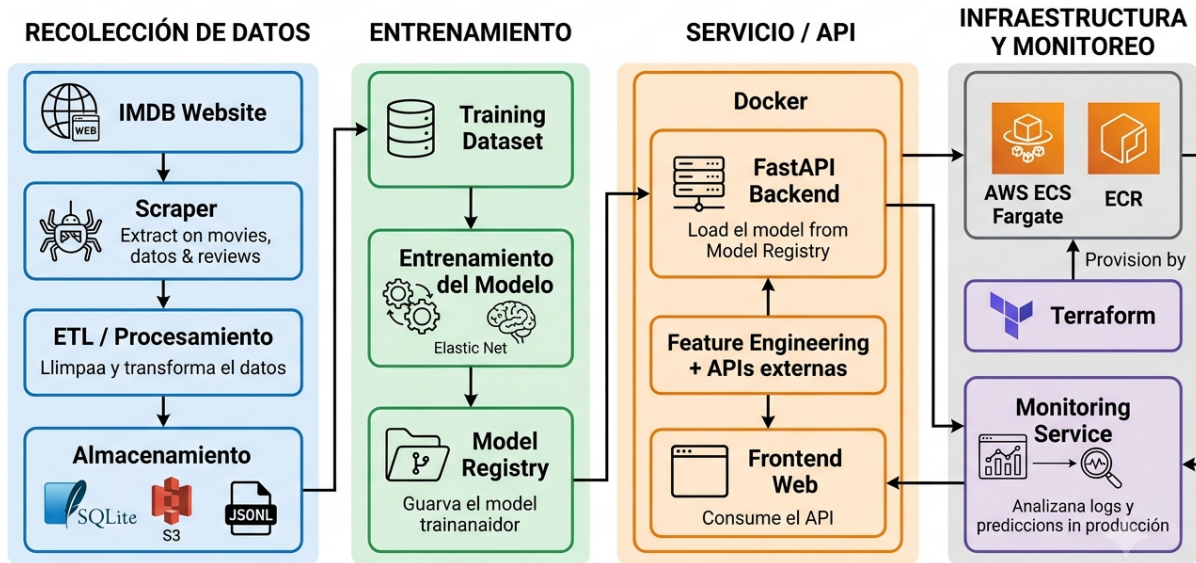


Figura 1: Arquitectura general de la solución: recolección de datos, entrenamiento, servicio/API e infraestructura cloud.

La Figura 1 resume la solución de punta a punta. El flujo comienza con la extracción de información desde IMDb mediante un scraper desarrollado en Python con Playwright. Ese scraper recolecta datos de películas y reviews, y los persiste en S3 como datos crudos, separados por tipo de entidad y por corrida. Esta primera capa funciona como data lake y permite conservar el historial de extracciones sin sobrescribir resultados anteriores.

Sobre esos datos crudos se ejecuta el proceso ETL, implementado con pandas y pyarrow. El ETL consolida las corridas del scraper, deduplica películas, agrega la señal de sentimiento de reviews y genera los datasets procesados que luego consumen el entrenamiento y la inferencia. De esta forma,

la arquitectura separa explícitamente la capa raw de la capa procesada, lo que mejora la trazabilidad y facilita repetir el procesamiento si cambia la lógica de transformación.

Luego, el pipeline de entrenamiento toma el dataset procesado, aplica feature engineering y entrena el modelo predictivo. Los experimentos, métricas y artefactos del modelo se registran en Weights & Biases, que cumple el rol de tracking y model registry. Desde allí, la aplicación de serving puede recuperar el modelo productivo usando el alias correspondiente.

La parte de serving está implementada con FastAPI y se empaqueta en una imagen Docker. Esa imagen se publica en ECR y se despliega en AWS ECS Fargate detrás de un Application Load Balancer. La infraestructura se define con Terraform, lo que permite recrear el entorno de manera declarativa. Finalmente, los logs de ejecución quedan disponibles en CloudWatch, completando una primera capa de observabilidad para la API desplegada.

4. Recolección de datos

El proceso de recolección y preparación inicial de datos se ejecutó localmente. Para hacerlo repetible, se creó un `Makefile` que agrupa los comandos principales de setup, credenciales, login en IMDb, ejecución del scraper y corrida del ETL. Esto evita depender de comandos manuales largos y permite correr el flujo completo de forma más ordenada.

Comando	Uso
<code>make install</code>	Instala las dependencias del proyecto con Poetry y descarga Chromium para Playwright.
<code>make aws-credentials</code>	Copia las credenciales del Learner Lab desde el portapapeles y las guarda en <code>~/.aws/credentials</code> .
<code>make scraper-login</code>	Abre el navegador para realizar el login manual en IMDb y guardar la sesión local.
<code>make scraper-run</code>	Ejecuta el scraper y sube los datos crudos a S3 en formato Parquet.
<code>make etl-run</code>	Lee los datos crudos desde S3, los procesa y publica el dataset de entrenamiento.

<code>make scraper-etl</code>	Ejecuta en orden el scraper y luego el ETL.
<code>make pipeline</code>	Ejecuta el flujo completo: credenciales, login, scraper y ETL.

Esta automatización fue especialmente útil porque el scraping requiere pasos locales, como la autenticación manual en IMDb, y pasos cloud, como la escritura de datos en S3 con credenciales temporales de AWS Academy/Learner Lab que expiran y deben renovarse periódicamente. El `Makefile` deja explícito el orden esperado, reduce errores operativos al repetir corridas y hace que el proceso sea más simple.

4.1. Ingesta de datos

La ingesta se realiza con un scraper de IMDb implementado en Python. El scraper obtiene información de películas y reviews, y guarda los resultados en formato local y en S3.

El scraper no parte de una lista fija de películas. En cada corrida genera IDs aleatorios dentro de un rango configurado, con el formato propio de IMDb, por ejemplo `tt00000001`. Con cada ID arma una URL de título:

```
https://www.imdb.com/title/{imdb_id}/
```

Luego visita esa página con Playwright, valida que el contenido corresponda a una película y extrae los atributos principales. Si el ID corresponde a otro tipo de contenido o no se encuentra título, se descarta y se continúa con otro ID aleatorio.

Las reviews se obtienen desde una URL aparte. Primero el scraper busca en la página de la película el enlace a la sección de reviews de usuarios; si no lo encuentra, usa por defecto:

```
https://www.imdb.com/title/{imdb_id}/reviews/
```

Esa página de reviews está paginada. Por eso el scraper lee las tarjetas visibles, expande spoilers cuando corresponde y luego intenta avanzar con el botón de “load more” hasta alcanzar el máximo configurado de reviews por película o hasta que ya no haya más páginas disponibles. Si

no logra obtener reviews desde la página completa, usa como fallback las reviews destacadas que aparecen dentro de la página principal de la película.

Los datos recolectados se separan en dos entidades principales:

Entidad	Campos recolectados
Películas	Identificador IMDb, título, año, rating IMDb, cantidad de votos, géneros, director/es, reparto principal, duración, certificado de edad, metascoring y sinopsis.
Reviews	Película asociada, usuario, rating de la review, texto de la review y votos de utilidad.

El scraper contempla que IMDb puede requerir login para acceder a reviews completas. Para eso existe un flujo manual que guarda una sesión local y permite reutilizarla en corridas posteriores.

4.2. Almacenamiento y versionado de datos

Los datos se almacenan en S3 en formato Parquet. Se eligió Parquet porque es un formato columnar, eficiente para lectura y escritura de datos tabulares, y se integra bien con pandas, pyarrow y servicios cloud como S3. Además, comprime mejor que formatos de texto como CSV o JSON y conserva tipos de datos de forma más robusta, algo útil para una capa raw que luego será procesada por el ETL.

Cada corrida del scraper genera archivos con timestamp, evitando sobrescribir datos anteriores.

```
s3://mlp-imdb-data/  
imdb/  
  movies/  
    scraped_date=YYYY-MM-DD/  
      run_YYYYMMDD_HHMMSS.parquet  
  reviews/  
    scraped_date=YYYY-MM-DD/
```

```
run_YYYYMMDD_HHMMSS.parquet
processed/
  training_dataset.json
inference/
  inference_dataset.json
```

Esta organización permite aplicar un concepto importante de MLOps: trazabilidad de datos. Si un modelo fue entrenado con determinada versión del dataset, se puede identificar de qué corridas del scraper provienen los datos.

4.3. ETL

El proceso ETL transforma los datos crudos del scraper en un dataset consolidado por película. En la implementación actual, el ETL lee datos desde S3, deduplica películas, agrega una señal de sentimiento a partir de reviews y genera un JSON procesado que luego consume el entrenamiento.

Los pasos principales son:

1. Validar que existan objetos en los prefijos `movies/` y `reviews/` de S3.
2. Leer todos los Parquet crudos de películas y reviews desde S3.
3. Deduplicar películas repetidas por `imdb_id`, conservando la versión con `scraped_date` más reciente.
4. Calcular `review_agg`, una tabla agregada de sentimiento promedio por película.
5. Unir películas con `review_agg` mediante `imdb_id`.
6. Eliminar filas sin `imdb_rating`, porque no pueden usarse para entrenamiento supervisado.
7. Generar registros limpios con tipos JSON válidos, rating normalizado a escala 0–1 y listas normalizadas.
8. Guardar `training_dataset.json` localmente y subirlo a S3:

```
s3://mlp-imdb-data/imdb/processed/training_dataset.json
```

Una de las transformaciones del ETL es generar `review_agg`. Como el scraper guarda varias reviews por película, el ETL resume esos textos en una única señal por `imdb_id`.

Para eso se aplica VADER, a través de la librería `vaderSentiment`. Se eligió porque es liviano, reproducible y no requiere entrenar un modelo adicional. VADER asigna un score de polaridad a cada review, y luego el ETL calcula el promedio por película.

Finalmente, esa señal agregada se une al dataset de películas. Si una película no tiene reviews válidas, simplemente queda sin esa feature y el pipeline de entrenamiento se encarga de imputarla.

El resultado del ETL es un JSON procesado con una fila por película y los campos necesarios para entrenamiento:

Campo del JSON procesado	Descripción
<code>imdb_id</code>	Identificador de IMDb; se conserva para trazabilidad, pero no entra al modelo.
<code>title</code>	Título de la película.
<code>year</code>	Año de estreno.
<code>votes</code>	Cantidad de votos.
<code>directors</code>	Lista normalizada de directores.
<code>main_cast</code>	Lista normalizada de actores principales.
<code>reviews</code>	Sentimiento promedio de reviews calculado con VADER y normalizado a escala 0–1.
<code>imdb_rating</code>	Variable objetivo, normalizada a escala 0–1.

4.4. Logs y diagnóstico del proceso

Tanto el scraper como el ETL incorporan logs para seguir la corrida completa y detectar problemas temprano. En el scraper se usa un logger propio que escribe tanto en consola como en archivos diarios dentro de `data/logs/`. Al comenzar, registra el rango de IDs aleatorios que va a intentar, la cantidad máxima de películas y la cantidad máxima de reviews por película. Esto permite reproducir el contexto de una corrida y entender si el volumen obtenido coincide con la configuración esperada.

Durante la autenticación, los logs indican si se reutilizó una sesión guardada, si fue necesario iniciar sesión nuevamente o si el login falló. Esto es importante porque, cuando IMDb bloquea el acceso a la página completa de reviews, el scraper puede quedar limitado a reviews destacadas. Registrar ese estado ayuda a explicar por qué una corrida puede traer menos reviews que otra.

Mientras recorre títulos, el scraper informa cuando descarta un ID porque no corresponde a una película, cuando no encuentra título, o cuando ocurre un error al extraer una película o sus reviews. También registra el progreso de guardado con el ID de IMDb, la cantidad de películas acumuladas y cuántas reviews se obtuvieron. Al final imprime una tabla con las películas guardadas ordenadas por cantidad de reviews y cuenta cuántas películas tienen rating pero no reviews. Esta salida sirve para análisis rápido: permite detectar películas sin señal textual, problemas de acceso a reviews o una muestra demasiado pobre para el ETL.

Finalmente, si S3 está configurado, el scraper registra la subida de los Parquet crudos con la cantidad de películas, cantidad de reviews y `run_id`. Si falla el upload, el log lo deja explícito, lo que facilita distinguir entre un problema de scraping y un problema de credenciales o conectividad con AWS.

El ETL incluye varios mensajes de diagnóstico para detectar problemas temprano y entender la calidad de los datos antes de entrenar. Primero valida que existan archivos en los prefijos de S3 para películas y reviews; si faltan datos o credenciales, muestra un mensaje específico para saber si el problema viene de AWS, del bucket o de una corrida pendiente del scraper.

Durante la lectura imprime la cantidad de filas crudas de películas y reviews. Después muestra

ejemplos útiles para análisis exploratorio: las 10 películas con mayor rating, las 10 con menor rating y las primeras reviews crudas. Estos logs permiten revisar rápidamente si los datos parecen razonables, si hay ratings fuera de escala o si las reviews llegan con los campos esperados.

Luego de deduplicar, el ETL informa cuántas películas quedan y muestra un resumen de nulos por dataset. Esto ayuda a identificar columnas incompletas, problemas del scraper o señales que luego habrá que imputar. Al final, informa cuántas filas sin `imdb_rating` se eliminaron, cuántas películas quedan en el dataset final y si todavía existen columnas con nulos. Con esa información se pueden sacar conclusiones simples antes de entrenar, por ejemplo si el dataset tiene pocas reviews útiles, si hay muchos registros sin target o si la extracción necesita repetirse.

4.5. Limitantes

Durante la recolección aparecieron algunas limitantes operativas. La primera fue que el scraping de reviews requería iniciar sesión en IMDb para acceder a la página completa de comentarios. Por eso se incorporó un paso de login manual y reutilización de sesión local; sin esa sesión, el scraper podía quedar limitado a las reviews destacadas disponibles en la página principal de la película.

Otra limitante fue la duración de las credenciales temporales de AWS Academy/Learner Lab. Como esas credenciales vencen en cuestión de algunas horas, las corridas se acotaron a volúmenes manejables, por ejemplo de a 500 reviews. Cuando se intentaba aumentar demasiado el volumen de scraping, podía pasar que al momento de guardar los datos en S3 las credenciales ya hubieran expirado, obligando a renovar credenciales y repetir parte del proceso.

Por último, al trabajar con una cuenta de Amazon Academy, no se logró dejar los datos de S3 accesibles para otros compañeros de forma simple. Esto limitó el uso compartido directo del bucket y reforzó la necesidad de documentar bien las rutas, corridas y archivos generados.

5. Entrenamiento

Esta sección describe el flujo completo de entrenamiento del modelo: cómo se transforman los datos procesados en features, cómo se entrena y evalúa el modelo, cómo se publica el artefacto

en W&B y qué controles se consideran para evitar problemas habituales en sistemas de Machine Learning en Producción.

5.1. Feature engineering

El feature engineering del modelo ocurre principalmente en `training/train.py`, después de cargar `training_dataset.json`. El ETL deja los datos limpios y agregados; el entrenamiento convierte esos campos en representaciones numéricas aptas para scikit-learn.

Uso de columnas en el entrenamiento actual:

Columna	Uso actual o motivo
<code>imdb_id</code>	Se excluye del entrenamiento porque es un identificador, no una señal predictiva.
<code>imdb_rating</code>	Se excluye de las features porque es el target.
<code>votes</code>	No se excluye en el modelo actual: se transforma a <code>log_votes</code> . Puede ser una fuente de leakage si se quiere simular una predicción previa al estreno o muy temprana, pero es válida para el escenario post-estreno.
<code>title</code>	No se excluye: se vectoriza con TF-IDF usando unigramas y bigramas.
<code>directors</code>	No se excluye: se convierte a <code>directors_text</code> y se codifica con <code>CountVectorizer</code> .
<code>main_cast</code>	No se excluye: se convierte a <code>cast_text</code> y se codifica con <code>CountVectorizer</code> .
<code>plot,</code> <code>runtime,</code> <code>certificate,</code> <code>metascore</code>	<code>genres,</code> El scraper puede recolectar estas columnas, pero no forman parte del <code>training_dataset.json</code> usado por el modelo actual. Quedan como trabajo futuro para mejorar el feature engineering.

5.1.1. Feature engineering del entrenamiento actual

El documento inicial del proyecto describe una versión de entrenamiento local basada en `training_dataset.json`. En esa versión, `train.py` aplica tres transformaciones principales antes de entrenar. Además, filtra registros sin director, convierte `reviews`, `year` y `votes` a valores numéricos, e imputa y escala las features numéricas dentro del pipeline.

La primera transformación es `log1p(votes)`. La cantidad de votos que recibe una película en IMDb tiene una distribución muy sesgada: unas pocas películas populares acumulan millones de votos, mientras que la mayoría tiene muchos menos. Esa asimetría perjudica a modelos lineales como ElasticNet. Aplicar `log1p` comprime la escala y hace que la variable sea más manejable sin perder poder predictivo.

La segunda transformación convierte las listas de directores y actores en representaciones vectoriales binarias. En el dataset original, `directors` y `main_cast` son listas de strings. Para usarlas en scikit-learn, se concatenan con el separador `|` y luego un `CountVectorizer` binario con un tokenizer por pipes genera una columna por director o actor que aparece al menos en dos películas. El resultado es un encoding multi-hot.

La tercera transformación vectoriza el título de la película con TF-IDF usando unigramas y bigramas. El título se trata como texto libre y se calculan pesos TF-IDF sobre los 1000 términos más frecuentes, considerando palabras individuales y pares de palabras consecutivas. Esto permite capturar patrones implícitos en el nombre de la película, como referencias a sagas, géneros o épocas.

Vale aclarar que esa versión del dataset no incluye algunas columnas que el scraper sí recolecta, como géneros, metascore, certificado de contenido o argumento. Esas features quedan fuera del modelo actual y representan una oportunidad clara de mejora para iteraciones futuras.

5.2. Entrenamiento del modelo

El entrenamiento actual toma como entrada el archivo local:

```
training/training_dataset.json
```

Como mejora MLOps, el entrenamiento debería leer ese dataset procesado desde S3 o registrar explícitamente qué versión local se utilizó, para mantener trazabilidad entre datos, corrida de entrenamiento y modelo publicado.

El dataset se separa en variables predictoras y target luego de construir las features derivadas:

```
features = [  
    "title",  
    "year",  
    "log_votes",  
    "reviews",  
    "directors_text",  
    "cast_text",  
]  
X = df[features]  
y = df["imdb_rating"]
```

Como primera aproximación se pueden usar modelos de regresión como Linear Regression, Random Forest Regressor, Gradient Boosting Regressor, XGBoost o LightGBM. En la implementación documentada inicialmente, el modelo entrenado es un ElasticNet dentro de un Pipeline de scikit-learn.

El script `train.py` lee la data procesada, aplica feature engineering y busca los mejores hiperparámetros `alpha` y `l1_ratio` mediante `GridSearchCV` con validación cruzada de 5 folds, optimizando MAE. Luego evalúa sobre un test set del 20% y reporta MAE, RMSE y R^2 . Además, imprime las mejores y peores predicciones individuales para facilitar el análisis de errores.

5.3. Publicación en W&B

La parte más importante del cierre del entrenamiento es la publicación en Weights & Biases [6]. El script consulta el MAE del modelo actualmente en producción antes de decidir si promover el nuevo modelo. Solo asigna el alias `production` si el nuevo modelo mejora al existente; en caso contrario, lo sube únicamente con `latest`. Esto evita que un reentrenamiento sobre los mismos datos

degrade el modelo productivo.

El equipo cuenta con un team en W&B, lo que permite centralizar los modelos fuera de cuentas personales y facilita que todos los integrantes puedan subir, consultar y promover artefactos.

Configuración del registry:

```
Entity: mlprod-obli
Project: imdb-rating
Artifact: imdb-rating-model
Alias: production
```

La idea es usar alias para promover modelos:

Alias	Uso
candidate	Modelo recién entrenado.
staging	Modelo validado para pruebas.
production	Modelo usado por la API.

5.4. Prevención de data leakage y training-serving skew

En un sistema de Machine Learning en Producción no alcanza con que el modelo tenga buenas métricas offline. También es necesario asegurar que el modelo no aprenda información que no estaría disponible al momento de predecir y que el procesamiento usado en entrenamiento sea el mismo que se usa en inferencia.

5.4.1. Data leakage

Data leakage ocurre cuando el modelo usa, directa o indirectamente, información del futuro o información demasiado cercana al target. Esto genera métricas artificialmente buenas durante entrenamiento, pero mal desempeño en producción.

Riesgo	Prevención
--------	------------

Usar <code>imdb_rating</code> como feature	Separar explícitamente X e y, dejando <code>imdb_rating</code> solo como target.
Usar <code>votes</code> como predictor	En el modelo actual se usa transformado como <code>log_votes</code> . Si la predicción representa una película nueva o temprana, debería excluirse o reemplazarse por una señal disponible en ese momento.
Usar reviews posteriores al momento de predicción	Definir fecha de corte o escenario de predicción.
Mezclar duplicados entre train y test	Deduplicar por <code>imdb_id</code> antes del split.
Hacer transformaciones mirando todo el dataset	Ajustar imputadores, escaladores y encoders solo con train.
Elegir modelo mirando el test final	Usar train/validation para selección y reservar test para evaluación final.

Para reviews, una decisión importante es distinguir entre predicción pre-estreno o temprana, donde no deberían usarse reviews de usuarios, y predicción post-estreno, donde pueden usarse siempre que se registre la fecha de corte.

5.4.2. Training-serving skew

Training-serving skew ocurre cuando los datos que recibe el modelo en producción no pasan por el mismo procesamiento que los datos usados para entrenar. Para reducir este riesgo, la aplicación en producción no reconstruye features manualmente. En su lugar, la UI muestra una lista de películas disponibles y, cuando el usuario selecciona una, la API obtiene los datos ya procesados desde `inference_dataset.json`.

Ese JSON de inferencia debería generarse con el mismo ETL que produce `training_dataset.json`. Por lo tanto, las columnas derivadas, codificaciones, agregaciones de reviews, tratamiento de nulos y tipos de datos nacen del mismo proceso usado para entrenar.

5.5. Trazabilidad de ML

La trazabilidad permite reconstruir qué datos, código, parámetros y modelo participaron en una predicción o en una versión publicada. En este proyecto se considera versionar experimentos, modelos y datos.

Elemento	Cómo se versiona	Para qué sirve
Experimentos	Corridas en W&B con parámetros, métricas y artefactos asociados	Comparar modelos, justificar la elección final y auditar resultados.
Modelos	Artifacts de W&B con versiones y aliases	Saber qué modelo está desplegado y volver a una versión anterior.
Datos crudos	Parquet en S3 particionados por fecha y corrida	Identificar de qué extracción provienen los datos.
Dataset procesado	<code>training_dataset.json</code> generado por el ETL	Relacionar cada entrenamiento con el dataset exacto utilizado.
Datos de inferencia	<code>inference_dataset.json</code> generado por el ETL	Asegurar consistencia entre entrenamiento e inferencia.

Para que la trazabilidad sea completa, cada corrida de entrenamiento debería guardar el dataset usado, fecha y versión del ETL, parámetros del modelo, métricas, commit de Git, artifact del modelo y lista de features esperadas.

6. Servicio/API

La capa de servicio expone el modelo entrenado para ser consumido desde una aplicación web y desde endpoints HTTP. Está implementada con FastAPI, carga el modelo productivo desde W&B y utiliza datos procesados para devolver predicciones de rating de forma consistente con el entrenamiento.

6.1. API de inferencia

La aplicación de serving está implementada con FastAPI [1]. Actualmente expone endpoints para:

Endpoint	Descripción
/	Interfaz web.
/status	Health check de la aplicación.
/movies	Lectura de películas desde S3.
/model	Consulta del artifact productivo en W&B.
POST /predict	Recibe una película seleccionada o su <code>imdb_id</code> , busca features en el JSON de inferencia y devuelve el rating estimado.

El endpoint de predicción carga el modelo productivo desde W&B, recupera del JSON de inferencia las features ya procesadas, excluye columnas que no deben usarse como entrada del modelo y devuelve una respuesta consistente.

6.2. Predicciones en landing page

Para hostear la landing page se optó por usar AWS ECS sobre AWS Academy [5]. Detrás de un Application Load Balancer se ejecuta una task construida desde una imagen publicada en AWS ECR, la cual se pushea localmente usando AWS CLI.

Antes de servir la imagen a ECR, se construye localmente a partir del `Dockerfile`. La imagen contiene el contenido estático de la página web y el web service que oficia de backend, utilizando JavaScript en el frontend y Python FastAPI como web engine.

Una vez levantada la task en ECS, la URL del Load Balancer expone el servicio en Internet. El servicio se conecta a W&B para descargar el modelo y hacer inferencia. Además, obtiene películas que ofician de input para la inferencia desde TMDB, sitio similar a IMDb que expone una API gratuita. Las películas se cargan a demanda desde una barra de búsqueda y, al seleccionar una película, se compara el score predicho contra el score real. Las películas se cargan a demanda en la barra de

búsqueda

Luego al seleccionar una película, se hace la predicción y se compara el score predicho contra el score real de la película en TDMB <imagen score>

6.3. Predicciones batch

Además de predicciones individuales por medio de la landing page, se deja planteado exponer predicciones en modo batch. Esta funcionalidad permitiría recibir un conjunto de películas, ejecutar inferencia sobre todas ellas y devolver un archivo o respuesta agregada con los ratings estimados. Para esto, el pod de ECS hostea un endpoint(/predict-batch) en donde dada una lista de identificadores de películas extraídos de TMDb, por ejemplo, "tmdb_ids" : [27205, 155, 550], el servicio retorna las predicciones para cada película incluyendo el rate real de la película en IMDB.

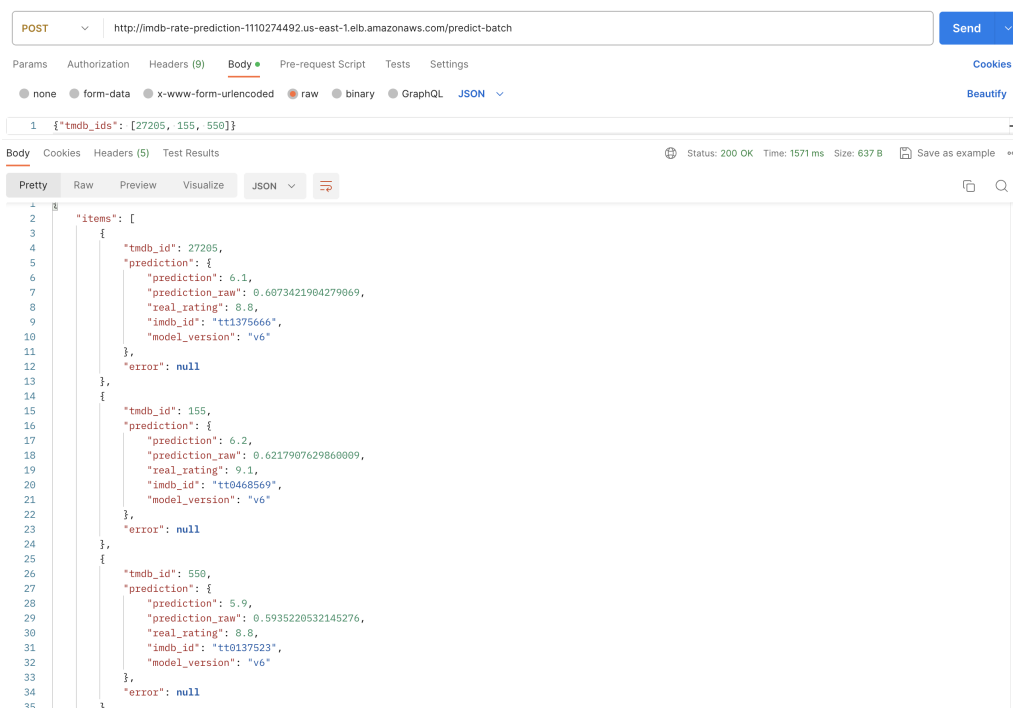


Figura 2: Predicción en Batch

6.4. Containerización

La aplicación se empaqueta con Docker usando una imagen base de Python. El contenedor instala las dependencias, copia la aplicación FastAPI y expone el puerto 8000. Esto permite ejecutar el mismo artefacto en local y en producción, reduciendo diferencias entre ambientes.

```
docker compose up --build
```

7. Infraestructura y monitoreo

La infraestructura productiva se define sobre AWS y busca que el despliegue sea repetible, observable y separado de la configuración sensible. En este bloque se agrupan la definición de infraestructura, la gestión de secretos, los mecanismos de monitoreo y la automatización operativa del proyecto.

7.1. Infraestructura como código

La infraestructura cloud se define con Terraform. Esto permite versionar los recursos, recrearlos y documentar la arquitectura de despliegue.

Recursos definidos:

- AWS ECS Cluster.
- Task Definition para Fargate.
- ECS Service.
- Application Load Balancer.
- Target Group con health check en `/status`.
- Security Groups.
- CloudWatch Log Group.

- Política de acceso a S3 para la task.

El despliegue usa una imagen Docker publicada en ECR y luego referenciada desde Terraform.

7.2. Gestión de secretos

La API necesita credenciales para consultar W&B. Para evitar hardcodear secretos, se usa AWS SSM Parameter Store.

```
wandb-org  
wandb-api-key
```

Si no se encuentran en SSM, la aplicación permite usar variables de entorno como fallback:

```
WANDB_USER  
WANDB_API_KEY
```

Esto aplica una buena práctica de seguridad: separar secretos del código fuente.

7.3. Observabilidad y monitoreo

La solución incluye un primer nivel de observabilidad:

- Endpoint `/status` como health check.
- Logs de contenedor enviados a CloudWatch.
- Consulta de versión del modelo productivo mediante `/model`.

Mejoras posibles:

- Registrar cantidad de predicciones realizadas.
- Medir latencia del endpoint `/predict`.
- Guardar errores de inferencia.

- Registrar distribución de features de entrada.
- Detectar drift comparando datos nuevos contra datos de entrenamiento.

8. Trabajo pendiente

Para completar la solución de punta a punta, quedan pendientes los siguientes puntos:

1. Documentar métricas finales del modelo productivo.
2. Incluir capturas del despliegue funcionando.
3. Completar la funcionalidad de predicciones batch.
4. Agregar tests básicos para ETL y API.
5. Registrar drift y métricas operativas del endpoint de predicción.

9. Limitaciones del proyecto y arquitectura ideal

9.1. Limitantes

Más allá de las limitantes puntuales ya mencionadas en la recolección de datos, el proyecto tuvo una restricción transversal que condicionó buena parte de las decisiones de arquitectura, se trabajó con cuentas individuales de AWS Academy/Learner Lab en lugar de una cuenta o una organización compartida entre los integrantes del equipo.

Esto trajo varias consecuencias prácticas:

- No fue posible compartir directamente buckets de S3, repositorios de ECR ni los servicios de ECS entre integrantes, ya que cada cuenta de Academy es aislada y temporal. Quien avanzaba con una parte de la infraestructura terminaba alojando esos recursos en su propia cuenta.
- El estado de Terraform no pudo centralizarse en un backend remoto compartido, por lo que

aplicar infraestructura quedó atado a la persona que la había creado originalmente.

- Los despliegues a ECR/ECS se hicieron de forma manual (`scripts/push-to-ecr.sh`, `terraform apply local`) en lugar de a través de un pipeline de CI/CD, porque no había un rol de despliegue ni credenciales de larga duración que todo el equipo pudiera usar de forma segura.
- Las credenciales temporales de Academy expiran en pocas horas, lo que además de limitar el volumen de scraping impidió dejar corriendo procesos programados (schedulers, triggers automáticos) que dependieran de esas credenciales.
- El gate de promoción de modelos que compara MAE contra producción (ver Publicación en W&B) corre hoy de forma manual/local al ejecutar `train.py`, y aunque promueva el alias `production` en W&B, la API en ECS solo carga el modelo una vez al iniciar (`deployment/main.py`), por lo que hace falta reiniciar el servicio a mano para que sirva el modelo nuevo.

9.2. Arquitectura ideal

La Figura 3 propone cómo debería quedar la arquitectura si el proyecto se llevara a un entorno real, siguiendo los principios de Continuous Delivery for Machine Learning (CD4ML) [2]. Los componentes marcados en azul con un check ya están implementados en la solución actual, mientras que los marcados en naranja con línea punteada son las piezas que faltarían para alcanzar una solución end to end.

IMDB Rating Prediction — MLOps Architecture (Current vs. Ideal / CD4ML)

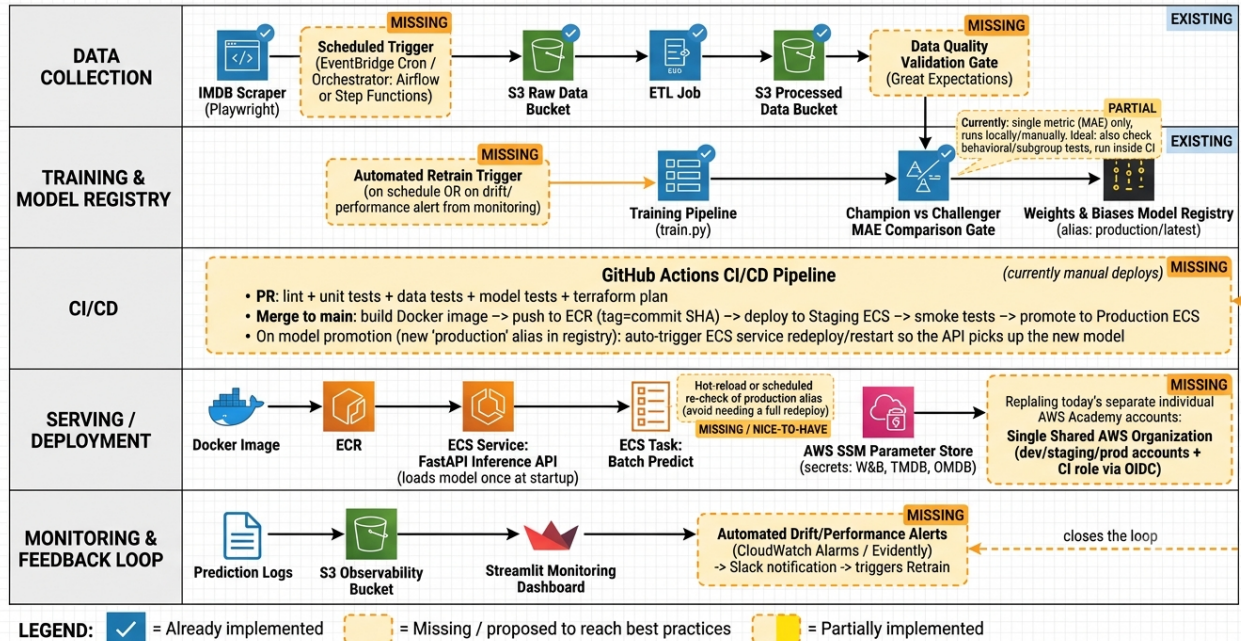


Figura 3: Arquitectura ideal propuesta (CD4ML): componentes ya implementados versus componentes faltantes para una solución MLOps completa.

A partir de esa comparación, las mejoras principales para un proyecto ideal serían:

- **Cuenta/organización de AWS compartida.** Reemplazar las cuentas individuales de Academy por una AWS Organization con cuentas separadas por ambiente (dev/staging/prod) y un rol de CI/CD asumible vía OIDC, de forma que el equipo completo pueda compartir buckets, ECR y el estado de Terraform sin depender de una única persona.
- **Recolección de datos programada.** Disparar el scraper y el ETL automáticamente (EventBridge, Step Functions o un orquestador como Airflow) en lugar de ejecutarlos manualmente, y agregar un gate de validación de calidad de datos (por ejemplo con Great Expectations) antes de habilitar un dataset para entrenar.
- **Reentrenamiento automático.** Disparar el pipeline de entrenamiento por schedule o por evento (nuevos datos suficientes, degradación detectada en el monitoreo), en lugar de requerir

que alguien lo corra a mano.

- **Fortalecer el gate de promoción de modelos.** El gate actual (comparar MAE contra el modelo en producción) es un buen punto de partida; se podría extender con tests de comportamiento y por subgrupos, y moverlo a un pipeline de CI en lugar de correr localmente.
- **Redeploy automático al promover un modelo.** Que la promoción del alias `production` en W&B dispare automáticamente un redeploy o reinicio del servicio de ECS, o alternatively que la API chequee periódicamente si hay un nuevo alias productivo, evitando servir un modelo desactualizado.
- **CI/CD real con GitHub Actions.** Lint, tests unitarios, tests de datos y de modelo, y `terraform plan` en cada PR; build de imagen, push a ECR con tag por commit, deploy a staging y promoción a producción automática al merge a main.
- **Cerrar el loop de monitoreo.** Convertir el dashboard de Streamlit, que hoy es de lectura manual, en una fuente de alertas automáticas (CloudWatch Alarms o herramientas como Evidently) que creen notificaciones y puedan disparar un reentrenamiento cuando se detecte drift o degradación de performance.

10. Conclusiones

El proyecto construye una base sólida para una solución de Machine Learning en Producción. No solo contempla el modelo, sino también los elementos necesarios para operarlo: datos versionados, procesamiento automatizado, empaquetado, infraestructura reproducible, despliegue cloud y registro de modelos.

Como cierre, la siguiente tabla resume los principales conceptos de clase aplicados en el proyecto:

Concepto	Aplicación en el proyecto
Pipeline de datos	Scraper + S3 + ETL.

Data lake	S3 como almacenamiento central.
Versionado de datos	Archivos particionados por fecha y corrida.
Feature engineering	Generación de features numéricas, categóricas y textuales.
Prevención de data leakage	Exclusión de variables no disponibles o demasiado cercanas al target.
Training-serving skew	Pipeline y schema compartidos entre entrenamiento e inferencia.
Reproducibilidad	Docker, Terraform y Makefile.
Tracking de experimentos	W&B para métricas y parámetros.
Model registry	Artifacts y alias en W&B.
Trazabilidad de ML	Versionado de experimentos, modelos, datos y features usadas.
Serving	FastAPI como API de inferencia.
CI/CD potencial	Build de imagen, push a ECR y deploy con Terraform.
Gestión de secretos	AWS SSM Parameter Store.
Observabilidad	Health check y CloudWatch logs.
Despliegue cloud	ECS Fargate + ALB.

La principal mejora para cerrar el ciclo MLOps completo es consolidar el entrenamiento productivo con el dataset procesado en S3, publicar métricas finales y terminar el flujo de predicciones batch. Aun así, la arquitectura actual ya refleja los componentes centrales de una solución MLOps: ingestión, procesamiento, entrenamiento, registry, serving, despliegue y observabilidad.

Referencias

- [1] FastAPI. Fastapi documentation. <https://fastapi.tiangolo.com/>, 2026.
- [2] Martin Fowler. Continuous delivery for machine learning. <https://martinfowler.com/articles/cd4ml.html>, 2016.
- [3] Chip Huyen. *Designing Machine Learning Systems*. O'Reilly Media, 2022.
- [4] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 2015.
- [5] Amazon Web Services. Amazon elastic container service documentation. <https://docs.aws.amazon.com/ecs/>, 2026.
- [6] Weights and Biases. Weights and biases documentation. <https://docs.wandb.ai/>, 2026.